Day 21 – LWC + Apex Integration, Dynamic Picklists & Interview Q&A

Learner: Ujjwal Nandy | **Object used:** `Service_Request__c` | **Status:** In progress (practice not yet completed)

Today you learn how a Lightning Web Component (LWC) talks to Apex: loading data with `@wire`, saving with imperative Apex, showing toasts, refreshing data with `refreshApex()`, and loading picklist values dynamically instead of hardcoding them.

1. The real-world scenario

A user opens a Salesforce page and wants to:

1. See the latest Service Requests.
2. Create a new Service Request.
3. Save it into Salesforce.
4. See the new record in the table without reloading the browser.
5. Get a clear success or error message.

Flow: LWC (HTML + JavaScript) → Apex controller (SOQL + validation + DML) → Salesforce database (`Service_Request__c`) → result returns to LWC.

2. Two ways to call Apex from LWC

Point	@wire	Imperative Apex
Meaning	Salesforce calls Apex and provisions data	You call Apex from JavaScript when you decide
Best use	Loading and displaying data	Save, update, delete, button clicks
Reactive	Yes	No, controlled by your code
Cacheable required	Yes (`cacheable=true`)	No
Returns	A stream of data to a property or function	A Promise

Point	@wire	Imperative Apex
DML allowed	No	Yes

Easy memory: **@wire = "load the data for me." Imperative = "call Apex now because the user clicked Save."**

3. Apex controller

File: `force-app/main/default/classes/ServiceRequestDay21Controller.cls`

```
public with sharing class ServiceRequestDay21Controller {

    // READ: latest 10 Service Requests (wire-friendly, so cacheable)
    @AuraEnabled(cacheable=true)
    public static List<Service_Request__c> getServiceRequests() {
        return [
            SELECT Id, Name, External_System_ID__c,
                   Subject__c, Category__c,
                   Priority__c, Status__c,
                   CreatedDate
            FROM Service_Request__c
            WITH USER_MODE
            ORDER BY CreatedDate DESC
            LIMIT 10
        ];
    }

    // WRITE: create a Service Request (imperative, NOT cacheable)
    @AuraEnabled
    public static Id createServiceRequest(Service_Request__c requestData) {

        if (requestData == null) {
            throw new AuraHandledException('Please provide Service Request
details.');
        }
        if (String.isBlank(requestData.Name)) {
            throw new AuraHandledException('Service Request Name is
required.');
        }
        if (String.isBlank(requestData.External_System_ID__c)) {
            throw new AuraHandledException('External System ID is
```

```
required.');
        }
        if (requestData.Account__c == null) {
            throw new AuraHandledException('Account is required.');
        }

        // Business rule: stamp the closed date when status is Closed
        if (requestData.Status__c == 'Closed') {
            requestData.Closed_Date__c = System.now();
        }

        try {
            insert as user requestData;
        } catch (DmlException e) {
            throw new AuraHandledException(e.getDmlMessage(0));
        }

        return requestData.Id;
    }
}
```

Understand the important lines

- `with sharing` respects record-level sharing rules (who can see which records).
- `@AuraEnabled(cacheable=true)` exposes a read method to LWC and allows client-side caching.
- `@AuraEnabled` (without cacheable) exposes a method that can do DML.
- `WITH USER_MODE` and `insert as user` enforce object permissions and field-level security (CRUD/FLS). Note: `with sharing` alone does **not** enforce CRUD/FLS.
- `AuraHandledException` sends a controlled, readable error message back to LWC.
- `return requestData.Id` returns the new record ID to JavaScript.

4. LWC component – dynamic picklists included

Create the folder `force-app/main/default/lwc/serviceRequestDay21` with three files.

File 1: serviceRequestDay21.html

```
<template>
    <lightning-card title="Day 21 - Service Request Manager">
        <div class="slds-p-around_medium">

            <h2 class="slds-text-heading_medium">Create Service Request</h2>

            <lightning-input label="Service Request Name" value={name}
                onchange={handleNameChange} required></lightning-input>

            <lightning-input label="External System ID" value={externalId}
                onchange={handleExternalIdChange} required></lightning-input>

            <lightning-input label="Account Record ID" value={accountId}
                onchange={handleAccountChange} required></lightning-input>

            <lightning-input label="Subject" value={subject}
                onchange={handleSubjectChange}></lightning-input>

            <lightning-combobox name="category" label="Category" value=
{category}
                options={categoryOptions} onchange={handleCategoryChange}>
            </lightning-combobox>

            <lightning-combobox name="priority" label="Priority" value=
{priority}
                options={priorityOptions} onchange={handlePriorityChange}>
            </lightning-combobox>

            <lightning-combobox name="status" label="Status" value={status}
                options={statusOptions} onchange={handleStatusChange}>
            </lightning-combobox>

            <lightning-button label="Save Service Request" variant="brand"
                onclick={handleSave} class="slds-m-top_medium" disabled=
{isSaving}>
            </lightning-button>

            <lightning-button label="Refresh List" onclick={handleRefresh}
                class="slds-m-top_medium slds-m-left_small">
            </lightning-button>
```

```
            <div class="slds-m-top_large">
                <h2 class="slds-text-heading_medium">Recent Service
Requests</h2>

                <template lwc:if={serviceRequests}>
                    <lightning-datatable key-field="Id" data=
{serviceRequests}
                        columns={columns} hide-checkbox-column>
                    </lightning-datatable>
                </template>

                <template lwc:if={errorMessage}>
                    <p class="slds-text-color_error">{errorMessage}</p>
                </template>
            </div>

        </div>
    </lightning-card>
</template>
```

File 2: serviceRequestDay21.js

```
import { LightningElement, wire } from 'lwc';
import { refreshApex } from '@salesforce/apex';
import { ShowToastEvent } from 'lightning/platformShowToastEvent';
import { getObjectInfo, getPicklistValues } from 'lightning/uiObjectInfoApi';

import getServiceRequests
    from '@salesforce/apex/ServiceRequestDay21Controller.getServiceRequests';
import createServiceRequest
    from
'@salesforce/apex/ServiceRequestDay21Controller.createServiceRequest';

import SERVICE_REQUEST_OBJECT from '@salesforce/schema/Service_Request__c';
import PRIORITY_FIELD from
'@salesforce/schema/Service_Request__c.Priority__c';
import STATUS_FIELD from '@salesforce/schema/Service_Request__c.Status__c';
import CATEGORY_FIELD from
'@salesforce/schema/Service_Request__c.Category__c';

export default class ServiceRequestDay21 extends LightningElement {
```

```
    // form values
    name = '';
    externalId = '';
    accountId = '';
    subject = '';
    category = '';
    priority = '';
    status = '';

    // dropdown options (filled dynamically)
    priorityOptions = [];
    statusOptions = [];
    categoryOptions = [];

    // table + state
    serviceRequests;
    errorMessage = '';
    isSaving = false;
    wiredRequestResult;

    columns = [
        { label: 'Name', fieldName: 'Name' },
        { label: 'External ID', fieldName: 'External_System_ID__c' },
        { label: 'Subject', fieldName: 'Subject__c' },
        { label: 'Category', fieldName: 'Category__c' },
        { label: 'Priority', fieldName: 'Priority__c' },
        { label: 'Status', fieldName: 'Status__c' }
    ];

    // ---------- Dynamic picklists ----------
    @wire(getObjectInfo, { objectApiName: SERVICE_REQUEST_OBJECT })
    objectInfo;

    @wire(getPicklistValues, {
        recordTypeId: '$objectInfo.data.defaultRecordTypeId',
        fieldApiName: PRIORITY_FIELD
    })
    wiredPriority({ data, error }) {
        if (data) {
            this.priorityOptions = this.toOptions(data);
        } else if (error) {
            console.error('Priority picklist error', error);
        }
```

```
    }

    @wire(getPicklistValues, {
        recordTypeId: '$objectInfo.data.defaultRecordTypeId',
        fieldApiName: STATUS_FIELD
    })
    wiredStatus({ data, error }) {
        if (data) {
            this.statusOptions = this.toOptions(data);
        } else if (error) {
            console.error('Status picklist error', error);
        }
    }

    @wire(getPicklistValues, {
        recordTypeId: '$objectInfo.data.defaultRecordTypeId',
        fieldApiName: CATEGORY_FIELD
    })
    wiredCategory({ data, error }) {
        if (data) {
            this.categoryOptions = this.toOptions(data);
        } else if (error) {
            console.error('Category picklist error', error);
        }
    }

    // convert Salesforce picklist metadata into combobox format
    toOptions(data) {
        return data.values.map(item => ({ label: item.label, value:
item.value }));
    }

    // ---------- Load table with @wire ----------
    @wire(getServiceRequests)
    wiredRequests(result) {
        this.wiredRequestResult = result;   // store the WHOLE result for
refreshApex
        if (result.data) {
            this.serviceRequests = result.data;
            this.errorMessage = '';
        } else if (result.error) {
            this.errorMessage = this.getErrorMessage(result.error);
        }
```

```
    }

    // ---------- Field handlers ----------
    handleNameChange(e) { this.name = e.target.value; }
    handleExternalIdChange(e) { this.externalId = e.target.value; }
    handleAccountChange(e) { this.accountId = e.target.value; }
    handleSubjectChange(e) { this.subject = e.target.value; }
    handleCategoryChange(e) { this.category = e.detail.value; }
    handlePriorityChange(e) { this.priority = e.detail.value; }
    handleStatusChange(e) { this.status = e.detail.value; }

    // ---------- Save with imperative Apex ----------
    async handleSave() {
        this.isSaving = true;

        const requestData = {
            Name: this.name,
            External_System_ID__c: this.externalId,
            Account__c: this.accountId,
            Subject__c: this.subject,
            Category__c: this.category,
            Priority__c: this.priority,
            Status__c: this.status
        };

        try {
            const recordId = await createServiceRequest({ requestData });

            this.showToast('Success',
                'Service Request created. Record ID: ' + recordId,
'success');

            this.clearForm();
            await refreshApex(this.wiredRequestResult);   // refresh the
table

        } catch (error) {
            this.showToast('Error', this.getErrorMessage(error), 'error');
        } finally {
            this.isSaving = false;
        }
    }
```

```
    async handleRefresh() {
        try {
            await refreshApex(this.wiredRequestResult);
            this.showToast('Success', 'Service Request list refreshed.',
'success');
        } catch (error) {
            this.showToast('Error', this.getErrorMessage(error), 'error');
        }
    }

    clearForm() {
        this.name = '';
        this.externalId = '';
        this.accountId = '';
        this.subject = '';
        this.category = '';
        this.priority = '';
        this.status = '';
    }

    getErrorMessage(error) {
        if (error && error.body) {
            if (Array.isArray(error.body)) {
                return error.body.map(item => item.message).join(', ');
            }
            if (error.body.message) {
                return error.body.message;
            }
        }
        return 'An unexpected error occurred.';
    }

    showToast(title, message, variant) {
        this.dispatchEvent(new ShowToastEvent({ title, message, variant }));
    }
}
```

File 3: serviceRequestDay21.js-meta.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">
    <apiVersion>YOUR_SUPPORTED_API_VERSION</apiVersion>
    <isExposed>true</isExposed>
    <targets>
        <target>lightning__AppPage</target>
        <target>lightning__HomePage</target>
        <target>lightning__RecordPage</target>
    </targets>
</LightningComponentBundle>
```

Replace `YOUR_SUPPORTED_API_VERSION` with the `sourceApiVersion` value in your `sfdx-project.json` (or any version supported by your org).

5. Dynamic picklist values – how it works

Hardcoding options means a developer must change code every time an admin adds a picklist value. Instead, Salesforce can provide the values:

1. `getObjectInfo` returns object metadata, including `defaultRecordTypeId`.
2. `getPicklistValues` takes that `recordTypeId` plus the field and returns the values.
3. We map them into `{ label, value }` objects for `lightning-combobox`.

```
@wire(getObjectInfo, { objectApiName: SERVICE_REQUEST_OBJECT })
objectInfo;

@wire(getPicklistValues, {
    recordTypeId: '$objectInfo.data.defaultRecordTypeId',
    fieldApiName: PRIORITY_FIELD
})
wiredPriority({ data, error }) { /* map data.values */ }
```

Key points

- The `$` in `'$objectInfo.data.defaultRecordTypeId'` makes the wire reactive. It waits until `objectInfo` has loaded, then runs automatically.
- Picklist values can differ per record type, which is why `recordTypeId` is required.
- If an admin adds a new value in Setup, the component shows it with no JavaScript change or redeployment.

- This approach is for **picklist fields only**.

Other dynamic dropdown scenarios

Scenario	Approach
Picklist field values	`getObjectInfo` + `getPicklistValues`
Records from an object (e.g. Account names)	Apex + SOQL, map to `{label, value}`
Searchable record selection	`lightning-record-picker`
Dependent picklists (controlling field)	`getPicklistValues` returns `controllerValues` and each value's `validFor`
Picklist per record type	Pass the specific `recordTypeId` instead of the default

Example – dropdown from records using Apex

```
@AuraEnabled(cacheable=true)
public static List<Account> getAccounts() {
    return [SELECT Id, Name FROM Account WITH USER_MODE ORDER BY Name LIMIT
50];
}
```

```
@wire(getAccounts)
wiredAccounts({ data, error }) {
    if (data) {
        this.accountOptions = data.map(acc => ({ label: acc.Name, value:
acc.Id }));
    }
}
```

This would let you replace the "Account Record ID" text box with a real Account dropdown (or use `lightning-record-picker`).

6. Deploy and test

Run these commands from your project root in the VS Code terminal.

Deploy the Apex class:

```
sf project deploy start --source-dir force-
app/main/default/classes/ServiceRequestDay21Controller.cls
```

Deploy the LWC:

```
sf project deploy start --source-dir force-
app/main/default/lwc/serviceRequestDay21
```

Open the org:

```
sf org open
```

(If you prefer the VS Code menu: right-click the file or folder → **SFDX: Deploy Source to Org**.)

Place the component on a page:

1. Setup → Lightning App Builder → New → App Page.
2. Find `serviceRequestDay21` under Custom Components and drag it onto the page.
3. Save and Activate.

7. Test scenarios (do all of them)

#	Scenario	Steps	Expected result
1	Load data	Open the page	Up to 10 latest records appear in the table
2	Dynamic dropdowns	Open Priority, Status, Category	Values match the picklists in Setup
3	New picklist value	Add a value in Setup → Object Manager → field, reload the page	New value appears with no code change
4	Successful save	Fill all required fields with a valid Account ID, click Save	Success toast, form cleared, new row in table
5	Missing Name	Leave Name blank, click Save	Error toast: Service Request Name is required

#	Scenario	Steps	Expected result
6	Missing External ID	Leave External ID blank	Error toast: External System ID is required
7	Missing Account	Leave Account blank	Error toast: Account is required
8	Invalid Account ID	Enter a wrong or non-Account ID	Error toast from Salesforce (invalid ID / cross-reference)
9	Duplicate External ID	Reuse an existing External System ID (if the field is unique)	Error toast with duplicate message
10	Closed request	Set Status = Closed and save	Record saves and `Closed_Date__c` is populated (verify on the record)
11	Refresh button	Click Refresh List	Info refreshed with success toast
12	Permissions	Log in as a user without edit access to the object	Save fails with a permission error (because of `insert as user`)

8. Troubleshooting

Problem	Likely cause	Fix
Component not in App Builder	`isExposed` is false or the target is missing	Check the meta XML and redeploy
Table empty	No records, or no field access	Check data and field-level security
Dropdown empty	Wrong field API name, or no FLS on the field	Verify the API names and permission set
Dropdown empty, no errors	`recordTypeId` not resolved	Check `getObjectInfo` loaded; check the console for errors
"Cannot find Apex method"	Class not deployed, or method not `@AuraEnabled`	Redeploy the class

Problem	Likely cause	Fix
Save error about method parameters	Parameter name mismatch	Key in JS must match the Apex parameter name (`requestData`)
Table not updating after save	Not storing the wired result	Store the whole `result` in `wiredRequestResult` and call `refreshApex` on it
`refreshApex` has no effect	Data came from an imperative call	`refreshApex` works only with wired data; call the method again for imperative data
Status value "Closed" not triggering date	API value differs from the label	Compare against the picklist **API value** in Apex
`Closed_Date__c` not set	Field is Date, not Date/Time	Use `System.today()` for Date fields

Debug tip: open browser DevTools (F12) → Console for JavaScript errors, and Setup → Debug Logs for Apex errors.

9. Key concepts to remember

- `refreshApex()` refreshes data provisioned by an **Apex @wire**. It does not directly refresh an imperative Apex result.
- `@wire` methods must be `cacheable=true` , which means they cannot do DML.
- Wired results are read-only. If you need to modify one, copy it first.
- Always store the **complete** wire result (not just `data`) if you want to call `refreshApex` later.
- `async` / `await` makes Promise-based code read like normal step-by-step code; `try/catch/finally` handles errors and cleanup (like re-enabling the Save button).
- Use `AuraHandledException` in Apex so users see a clean message instead of a generic one.
- Enforce CRUD/FLS in production with `WITH USER_MODE` , `insert as user` , or `Security.stripInaccessible()` .

10. Interview questions and answers

Q1. What are the two ways to call Apex from LWC? Using `@wire` and imperative Apex. `@wire` automatically provisions data, mainly for reads. Imperative Apex is called from JavaScript when we choose, such as on a button click.

Q2. What is the difference between @wire and imperative Apex? `@wire` is reactive, automatic, requires a cacheable method and cannot perform DML. Imperative is manual, returns a Promise, works for read and write methods, and does not require `cacheable=true`.

Q3. Why must a wired Apex method be cacheable? Wire is designed for reading data, and Salesforce can cache and re-provision it. Cacheable methods cannot perform DML, which keeps reads safe and predictable.

Q4. What is `refreshApex()`? It re-fetches data previously provisioned by an Apex wire adapter, bypassing the cache. After creating a record, I call `refreshApex(this.wiredRequestResult)` so the table shows the latest data.

Q5. Why do you store the whole wire result and not only `data`? `refreshApex` needs the original wire result object (which includes `data` and `error`) to know which wire to refresh.

Q6. How do you handle errors in LWC and Apex? In Apex I validate input and throw `AuraHandledException` with a friendly message. In LWC I use `try/catch` for imperative calls, check `result.error` for wired calls, and show messages with `ShowToastEvent`.

Q7. What is `AuraHandledException` and why use it? It lets Apex return a controlled, user-readable error message to the Lightning component. A normal exception shows a generic "script-thrown exception" message.

Q8. Can an Apex method be called from LWC without `@AuraEnabled`? No. The method must be `@AuraEnabled` and `static` (and public or global).

Q9. Can we use `@wire` to insert records? No. Wired methods are read-only and cacheable. Use imperative Apex for insert, update and delete (or `lightning/uiRecordApi` for simple record operations).

Q10. What does `with sharing` do? Does it enforce field-level security? It enforces record-level sharing rules for the running user. It does not enforce CRUD or FLS. For that, use `WITH USER_MODE`, `as user` DML, or `Security.stripInaccessible()`.

Q11. What are `async` and `await`? `async` marks a function that returns a Promise. `await` pauses execution inside it until the Promise resolves, so the code after it runs only when the result is ready.

Q12. What is a Promise? An object that represents the eventual success or failure of an asynchronous operation. Imperative Apex returns a Promise.

Q13. How do you populate a picklist dynamically in LWC? I use `getObjectInfo` to get the default record type ID, then `getPicklistValues` with that ID and the field. I map `data.values` to `{ label, value }` and bind them to `lightning-combobox`.

Q14. Why is `recordTypeId` needed for `getPicklistValues`? Picklist values can differ by record type, so Salesforce needs to know which record type's values to return. If the object has no custom record types, the default (master) record type is used.

Q15. What does the `$` mean in `'$objectInfo.data.defaultRecordTypeId'`? It makes the wire parameter reactive. The wire waits until the value is available and re-runs when it changes.

Q16. What happens if an admin adds a new picklist value? The component shows the new value automatically after the metadata refreshes, with no JavaScript change or redeployment.

Q17. How would you build a dropdown from another object's records? Write a cacheable Apex method with SOQL, return the records, and map them in JavaScript to `{ label: record.Name, value: record.Id }`. For searching large sets, use `lightning-record-picker`.

Q18. How do you handle dependent picklists dynamically? `getPicklistValues` for the dependent field returns `controllerValues` and `validFor` indexes on each value. Filter the dependent options based on the controlling field's selected value.

Q19. Why disable the Save button while saving? To prevent double submissions (duplicate records) while the Apex call is still running. Re-enable it in the `finally` block.

Q20. How do you pass an object from LWC to an Apex method? Pass a JavaScript object whose keys match the Apex parameter name and field API names: `createServiceRequest({ requestData: { Name: 'X', ... } })`.

Q21. Why did your Closed_Date__c logic go in Apex and not in LWC? Business rules belong on the server so they apply no matter which client (LWC, API, integration) creates the record. Client code can be bypassed.

Q22. Where do you import an Apex method in LWC? `import methodName from '@salesforce/apex/ClassName.methodName';`

11. Understanding check – answers

1. Which annotation makes an Apex read method available to LWC with caching? **`@AuraEnabled(cacheable=true)`**
2. Which method suits automatically loading records? **`@wire`**
3. What does `List<Service_Request__c>` mean? **Multiple Service Request records**
4. Where do you import an Apex method in LWC? **`@salesforce/apex/ClassName.MethodName`**
5. Which API gives you picklist values? **`getPicklistValues`** **from `lightning/uiObjectInfoApi`**
6. What refreshes wired Apex data after a save? **`refreshApex(this.wiredRequestResult)`**

12. Practice assignment checklist

Tick these off as you finish them:

- [ ] Create the Apex controller and deploy it.
- [ ] Create the LWC component (html, js, meta xml) and deploy it.
- [ ] Display the latest 10 Service Requests in the table.
- [ ] Load Priority, Status and Category dynamically with `getPicklistValues`.
- [ ] Create a Service Request using imperative Apex.
- [ ] Show a success toast after saving.
- [ ] Show an error toast when required fields are missing.
- [ ] Refresh the table using `refreshApex()`.
- [ ] Test a Closed Service Request and verify `Closed_Date__c`.
- [ ] Add a new picklist value in Setup and confirm it appears without code changes.
- [ ] Replace the Account ID text box with an Account dropdown or `lightning-record-picker` (stretch goal).

13. Next step

Finish the checklist in your org, then share your JavaScript or any error message and I will review it like a Salesforce technical interviewer.

Day 22 – LWC Advanced Concepts: parent–child communication revision, `@api`, `@track` and reactive properties, getters and setters, conditional rendering, dynamic forms and validation, and real-world interview questions.